

Unit IV Class Activity

1. A university wants to determine whether students who spend more hours studying obtain higher marks.

Dataset

StudyHours Marks

2	45
3	52
4	58
5	63
6	69
7	74
8	81
9	88

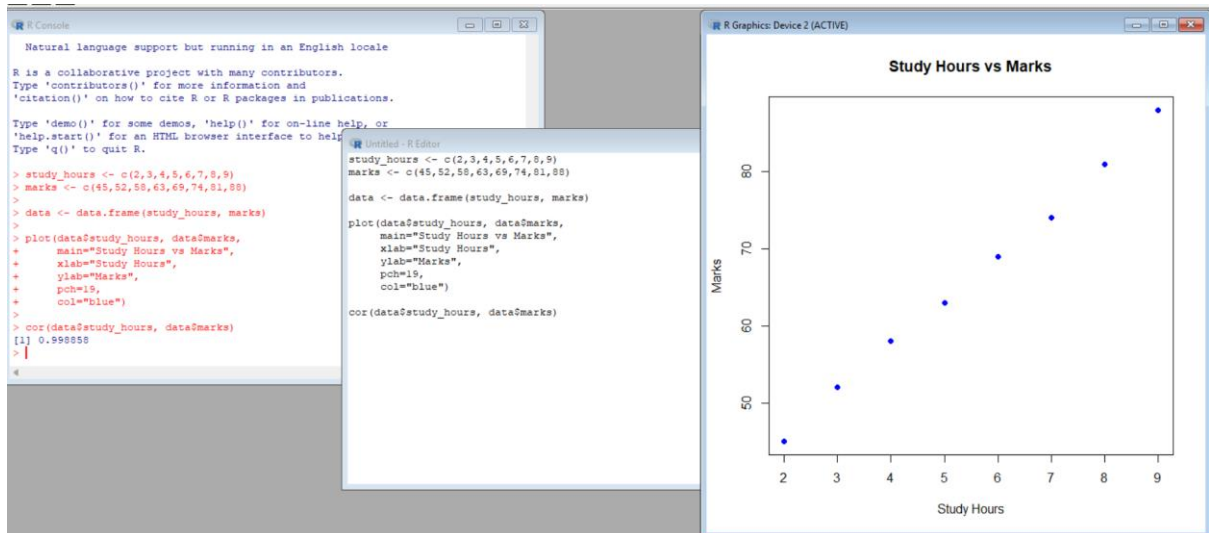
Task

- Create the dataset in R.
- Draw a scatterplot.
- Add axis labels and title.
- Interpret whether a positive correlation exists.

A. CODE

```
study_hours <- c(2,3,4,5,6,7,8,9)
marks <- c(45,52,58,63,69,74,81,88)
data <- data.frame(study_hours, marks)
plot(data$study_hours, data$marks,
      main="Study Hours vs Marks",
      xlab="Study Hours",
      ylab="Marks",
      pch=19,
      col="blue")
cor(data$study_hours, data$marks)
```

OUTPUT:



2. A hospital records patients' age and systolic blood pressure.

Write a Python program using Matplotlib to:

Create a scatterplot.

Display gridlines.

Change point color to red.

Comment on the observed trend.

A. CODE

```

import matplotlib.pyplot as plt

age = [20,25,30,35,40,45,50,55,60]

bp = [110,115,118,122,128,132,138,145,150]

plt.scatter(age, bp, color="red")

plt.xlabel("Age")

plt.ylabel("Systolic Blood Pressure")

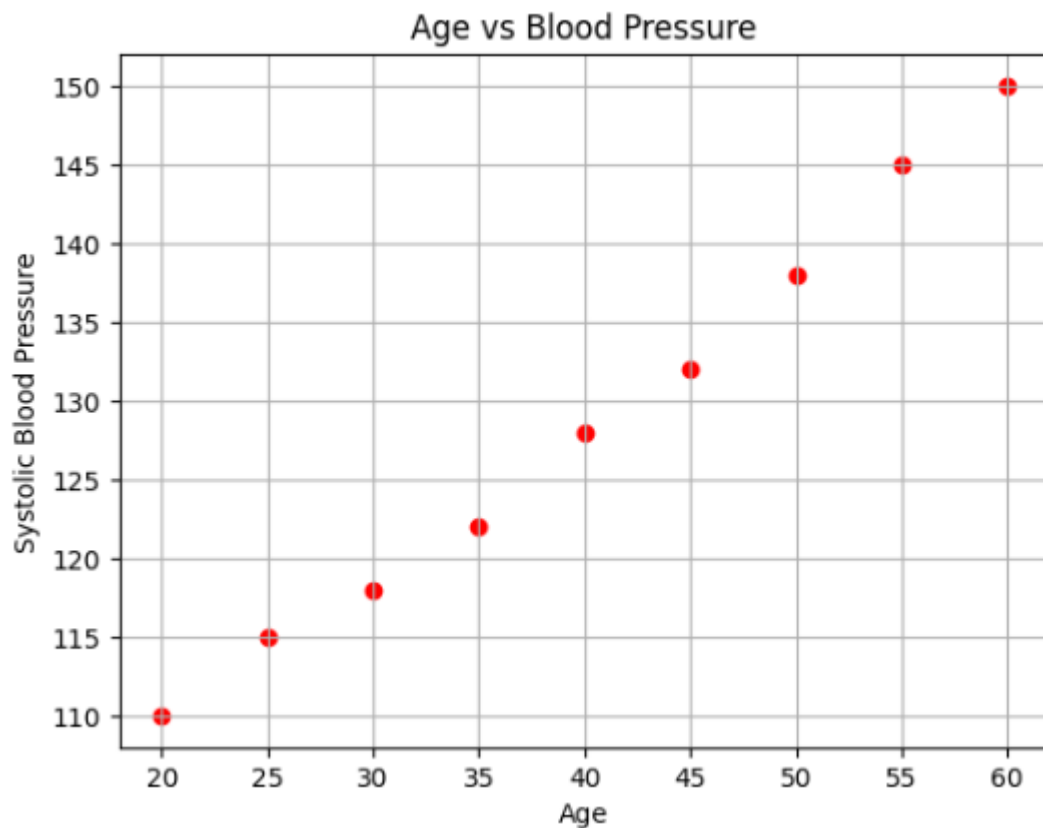
plt.title("Age vs Blood Pressure")

plt.grid(True)

plt.show()

```

OUTPUT:



3. A company wants to investigate whether advertising expenditure influences monthly sales. Develop a scatterplot and determine whether sales increase with advertising expenditure.

A financial institution has collected information on:

Income

Savings

Loan Amount

Credit Score

Generate a correlation matrix and visualize it using a correlogram.

Which variables exhibit the strongest positive correlation?

A. CODE

```
import matplotlib.pyplot as plt

advertising = [10,20,30,40,50,60,70,80]
sales = [100,120,135,150,170,185,205,225]

plt.scatter(advertising, sales)

plt.xlabel("Advertising Expenditure")

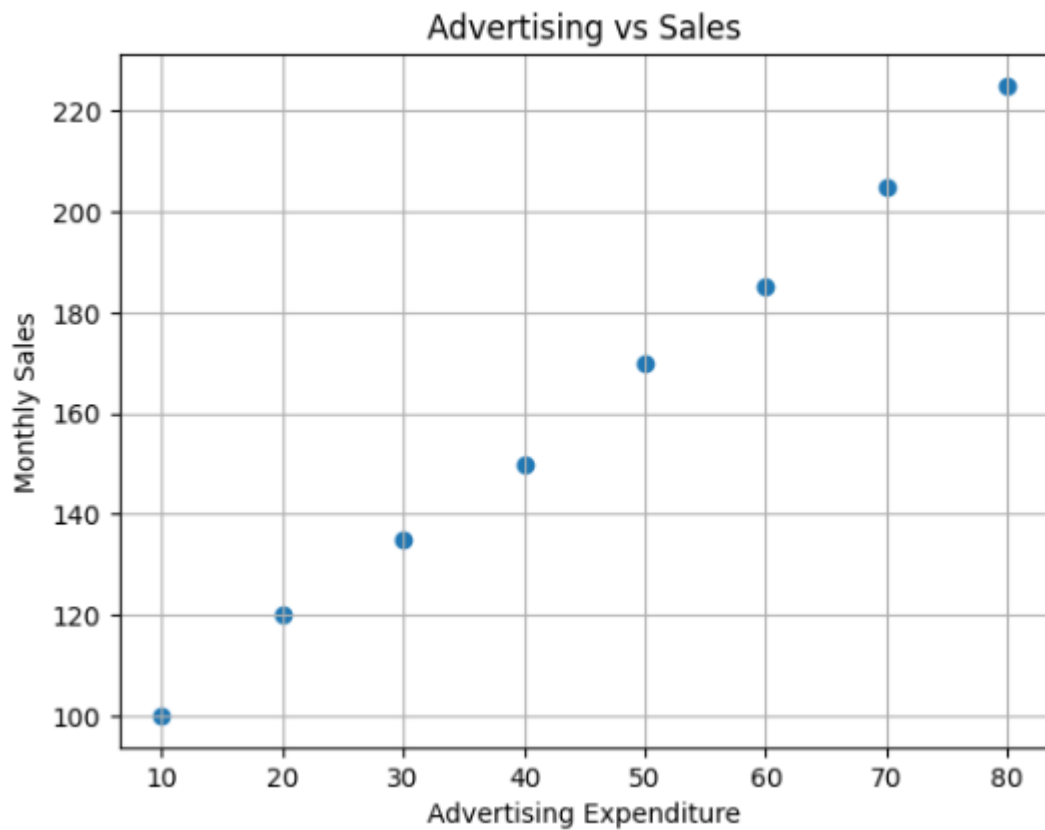
plt.ylabel("Monthly Sales")
```

```
plt.title("Advertising vs Sales")
```

```
plt.grid(True)
```

```
plt.show()
```

OUTPUT



4. A financial institution has collected information on:

Income

Savings

Loan Amount

Credit Score

Generate a correlation matrix and visualize it using a correlogram.

Which variables exhibit the strongest positive correlation?

A. CODE

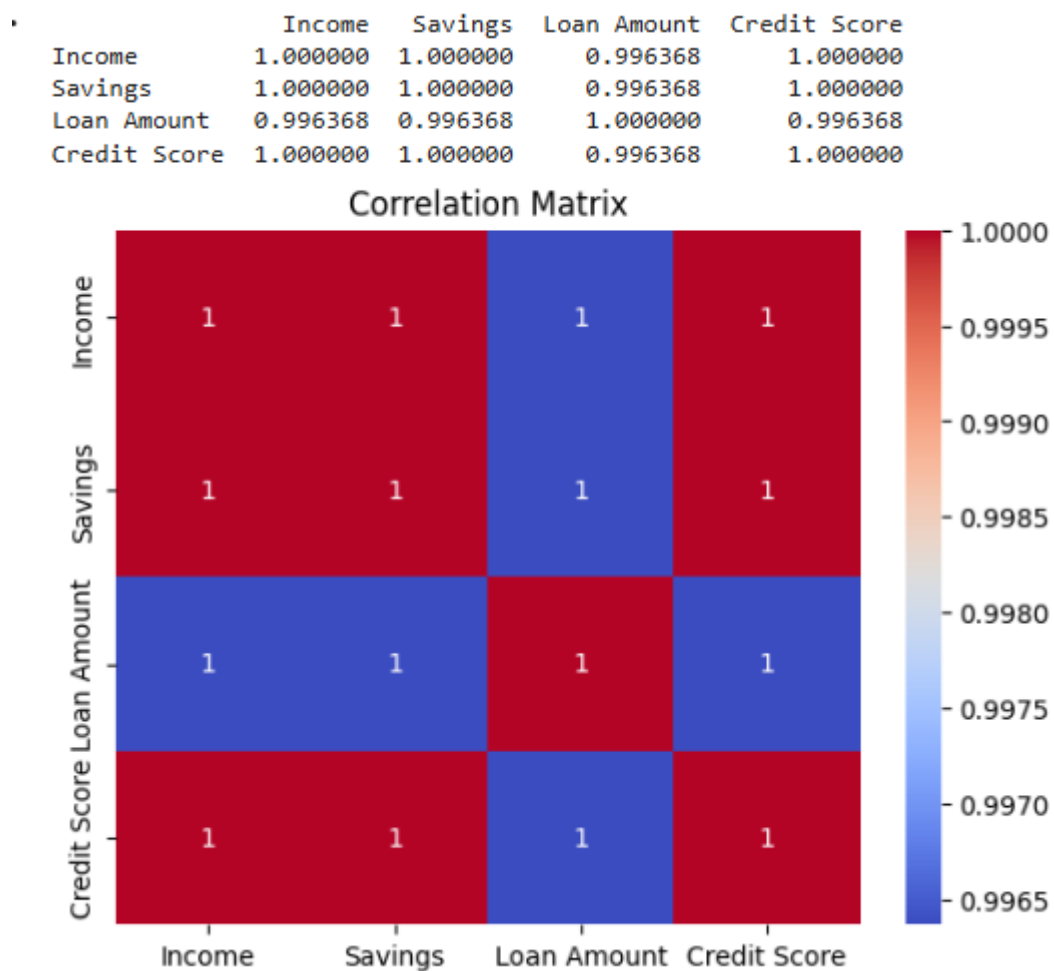
```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = pd.DataFrame({
    "Income":[30,40,50,60,70,80,90,100],
    "Savings":[10,15,20,25,30,35,40,45],
    "Loan Amount":[20,25,30,40,45,50,60,65],
    "Credit Score":[650,670,690,710,730,750,770,790]
})
corr = data.corr()
print(corr)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

OUTPUT



- Using the Iris dataset,
Compute the correlation matrix.

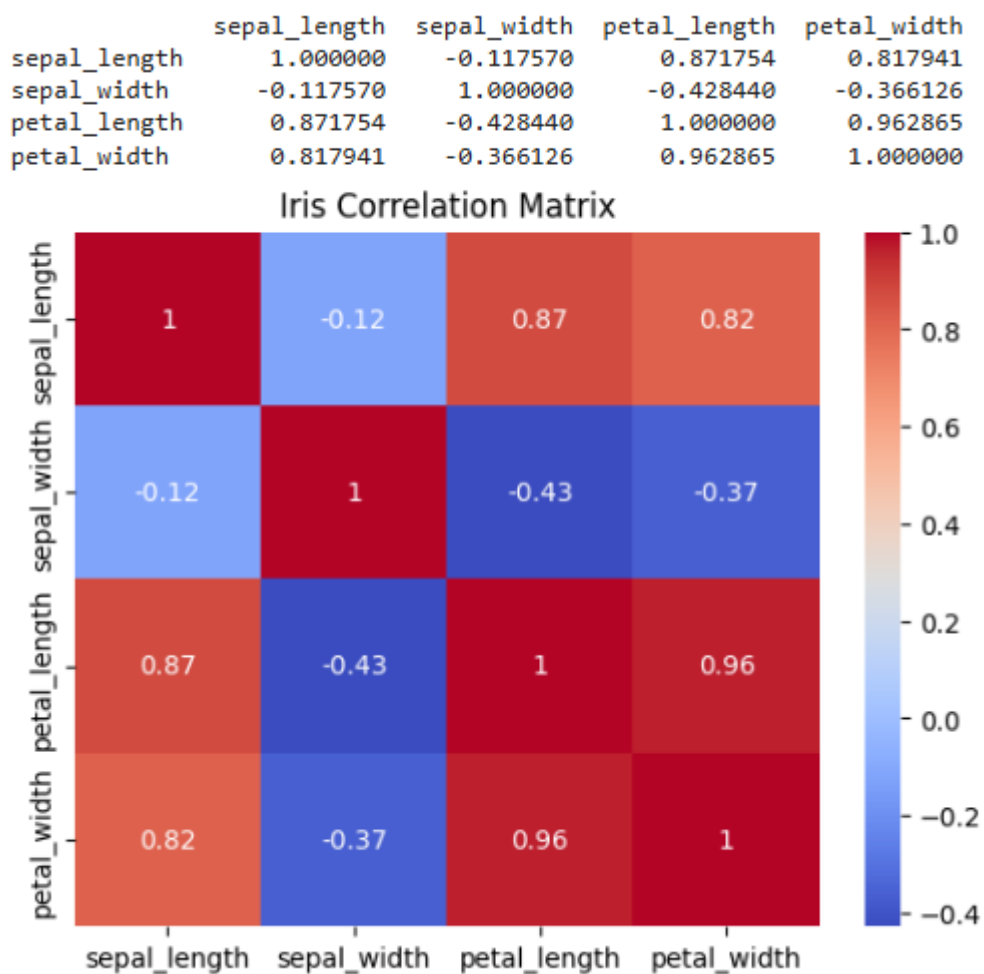
Display it as a heatmap using Seaborn.

Identify variables having negative correlation.

A.CODE

```
import seaborn as sns
import matplotlib.pyplot as plt
iris = sns.load_dataset("iris")
corr = iris.iloc[:, :4].corr()
print(corr)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Iris Correlation Matrix")
plt.show()
```

OUTPUT:



6. A healthcare dataset contains:

BMI

Cholesterol

Blood Pressure

Glucose

Heart Rate

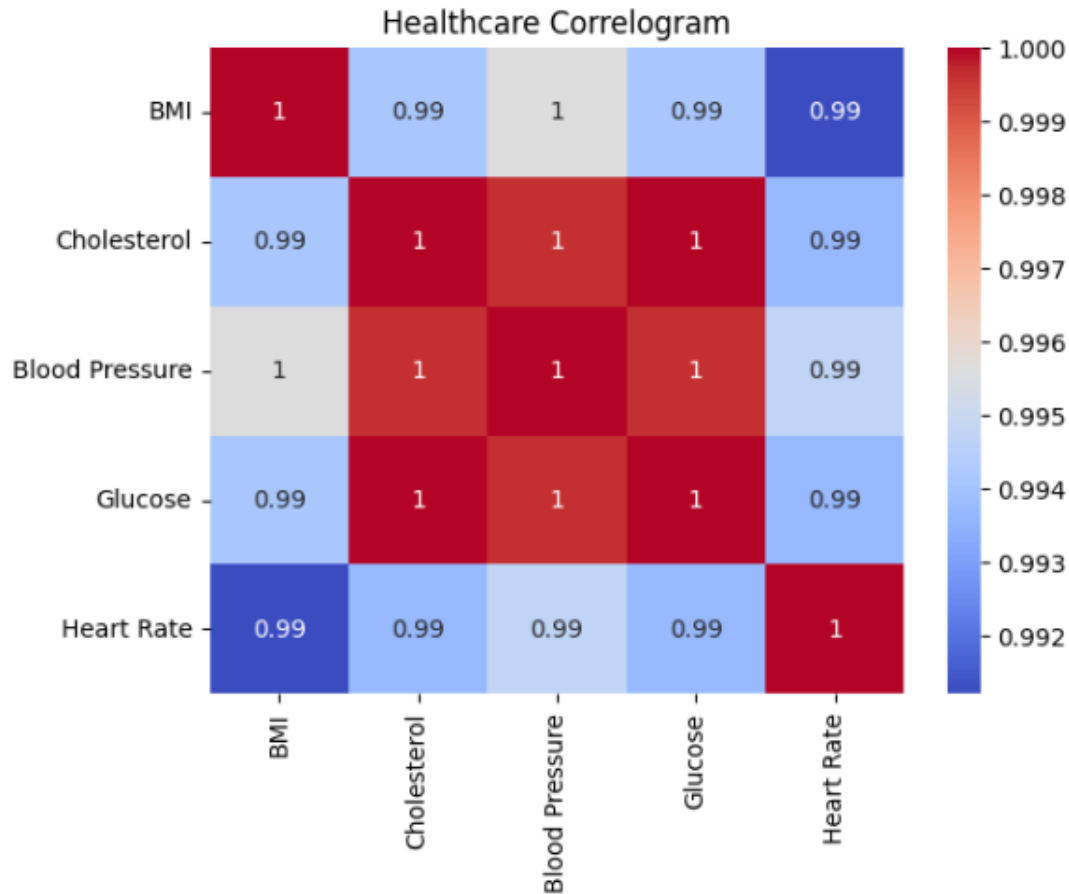
Create a correlogram and identify multicollinearity among variables.

A.CODE

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data = pd.DataFrame({
    "BMI":[22,25,28,30,32,35,27,29],
    "Cholesterol":[180,190,200,210,220,230,195,205],
    "Blood Pressure":[110,115,120,125,130,135,118,123],
    "Glucose":[80,90,100,110,120,130,95,105],
    "Heart Rate":[65,68,72,75,78,80,70,74]
})
corr = data.corr()
print(corr)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Healthcare Correlogram")
plt.show()
```

OUTPUT

	BMI	Cholesterol	Blood Pressure	Glucose	Heart Rate
BMI	1.000000	0.994135	0.995614	0.994135	0.991214
Cholesterol	0.994135	1.000000	0.999597	1.000000	0.993695
Blood Pressure	0.995614	0.999597	1.000000	0.999597	0.994729
Glucose	0.994135	1.000000	0.999597	1.000000	0.993695
Heart Rate	0.991214	0.993695	0.994729	0.993695	1.000000



7. A researcher is analyzing 25 features extracted from MRI images. Perform PCA to reduce the dimensionality to two principal components. Plot the transformed data.

A.CODE

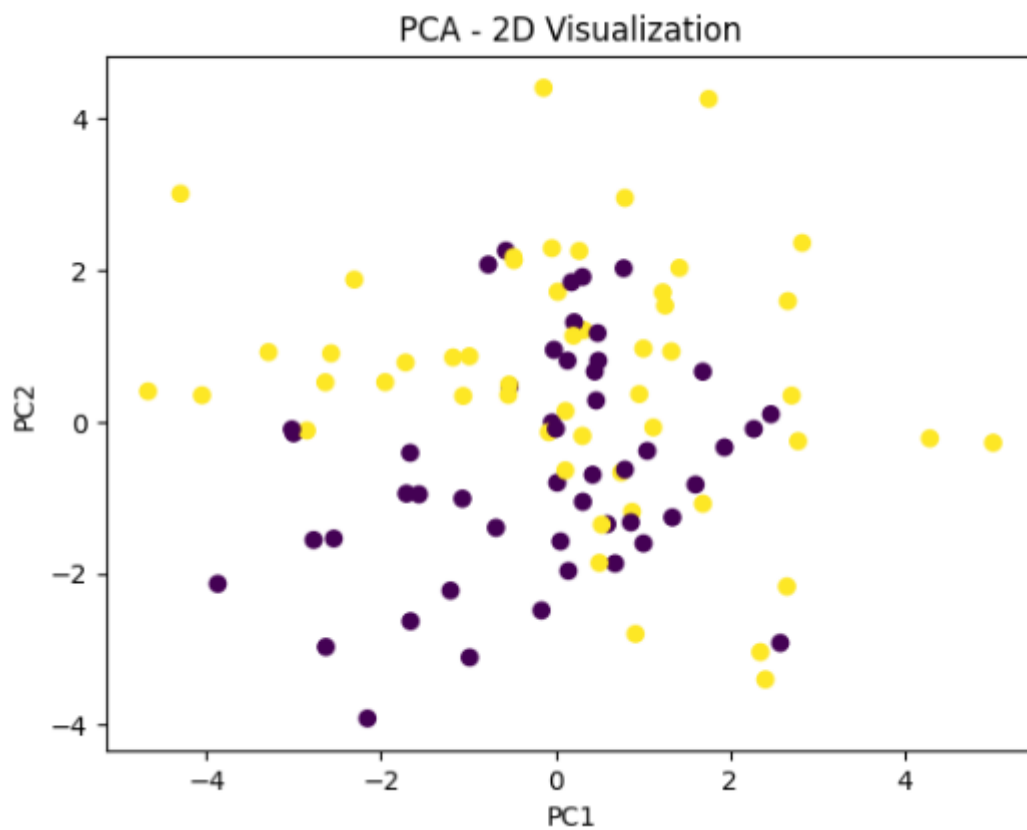
```
from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

X, y = make_classification(
    n_samples=100,
    n_features=25,
    n_informative=15,
```

```

random_state=42
)
X = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA - 2D Visualization")
plt.show()

```



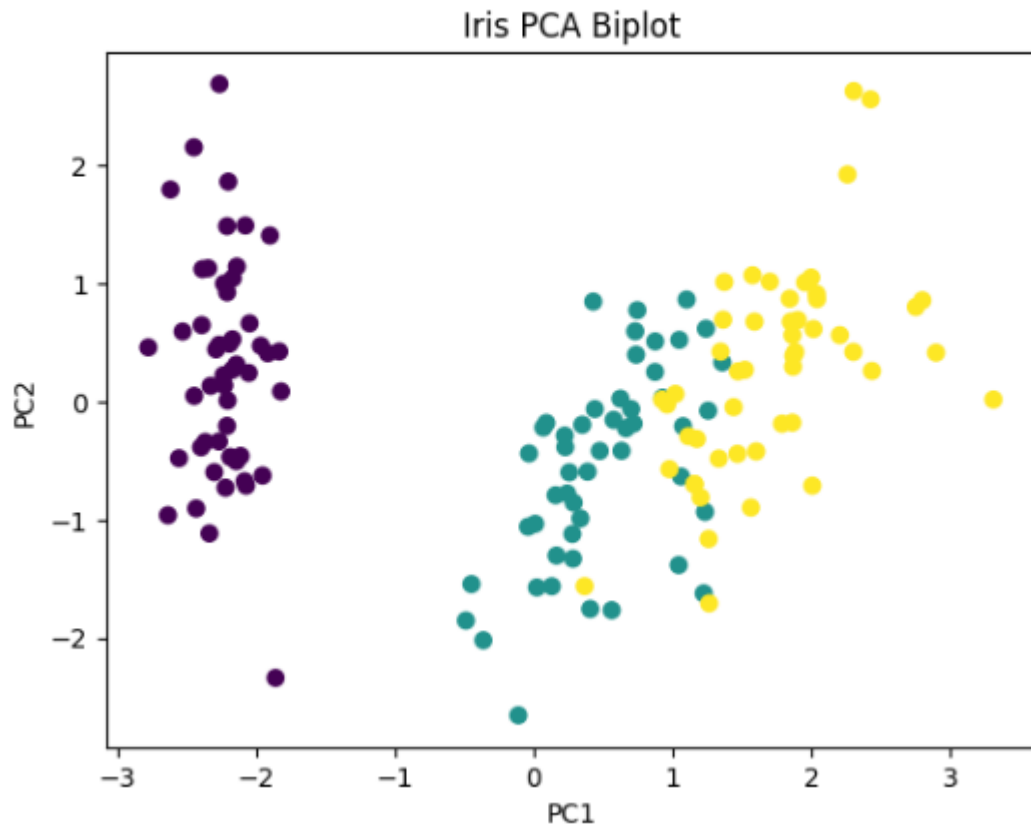
8. Using the Iris dataset,
Perform PCA.
Display the summary.
Draw a biplot.
Explain which principal component explains the highest variance.
- A. CODE

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

iris = load_iris()
X = StandardScaler().fit_transform(iris.data)
pca = PCA()
X_pca = pca.fit_transform(X)
print(pca.explained_variance_)
print(pca.explained_variance_ratio_)
plt.scatter(X_pca[:,0], X_pca[:,1], c=iris.target)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Iris PCA Biplot")
plt.show()
```

OUTPUT

```
• [2.93808505 0.9201649 0.14774182 0.02085386]  
  [0.72962445 0.22850762 0.03668922 0.00517871]
```

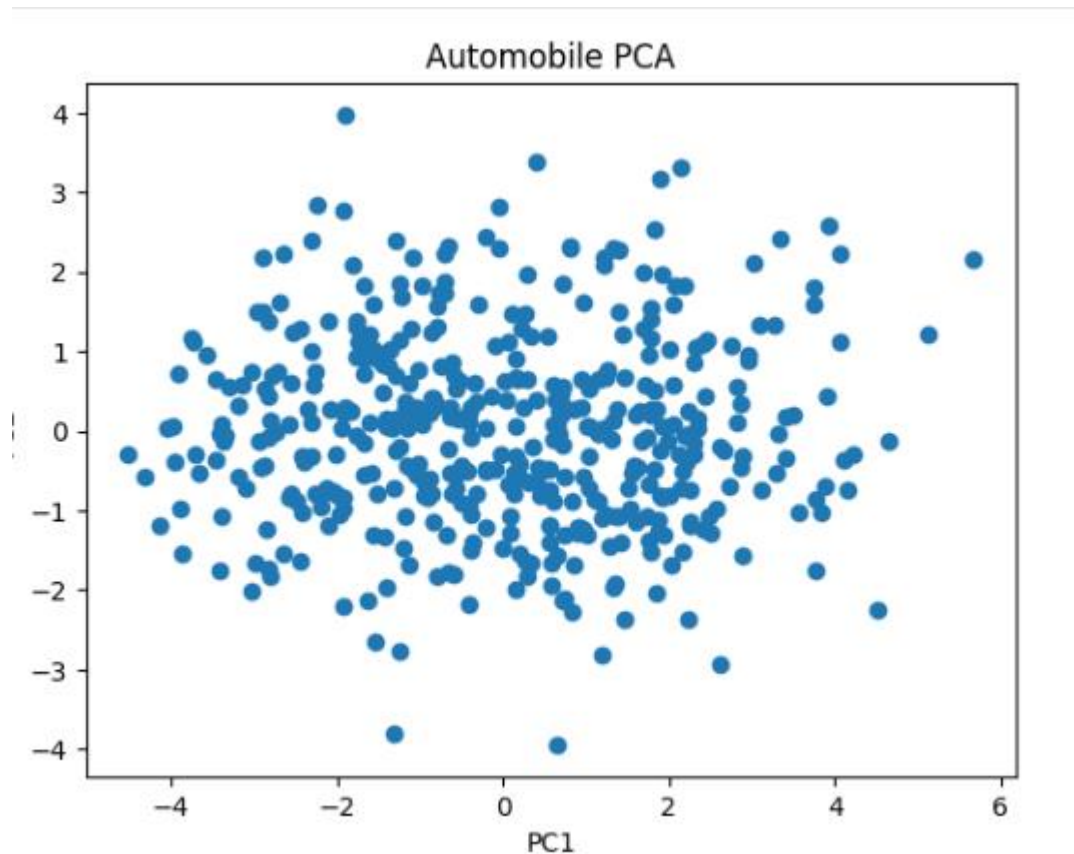


9. An automobile company has recorded 15 specifications for various cars. Reduce the dimensions using PCA and visualize the first two principal components.

A.CODE

```
from sklearn.datasets import load_diabetes  
from sklearn.preprocessing import StandardScaler  
from sklearn.decomposition import PCA  
import matplotlib.pyplot as plt  
data = load_diabetes()  
X = StandardScaler().fit_transform(data.data)  
pca = PCA(n_components=2)  
X_pca = pca.fit_transform(X)  
plt.scatter(X_pca[:,0], X_pca[:,1])  
plt.xlabel("PC1")  
plt.ylabel("PC2")  
plt.title("Automobile PCA")
```

plt.show()



10. A facial recognition dataset contains 128-dimensional facial embeddings.

Use t-SNE to visualize the data in two dimensions.

Explain why t-SNE is preferred over PCA for visualization.

CODE

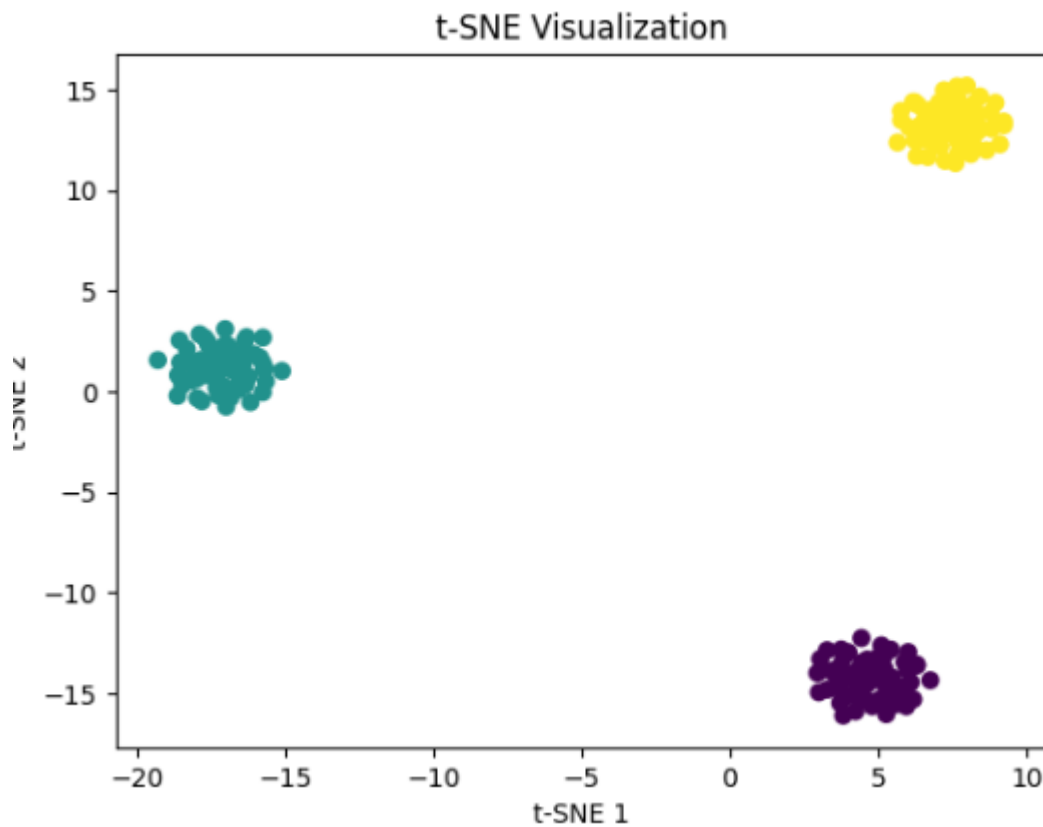
```
from sklearn.datasets import make_blobs
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

X, y = make_blobs(
    n_samples=200,
    n_features=128,
    centers=3,
    random_state=42
)

X_tsne = TSNE(n_components=2, random_state=42).fit_transform(X)
plt.scatter(X_tsne[:,0], X_tsne[:,1], c=y)
```

```
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.title("t-SNE Visualization")
plt.show()
```

OUTPUT



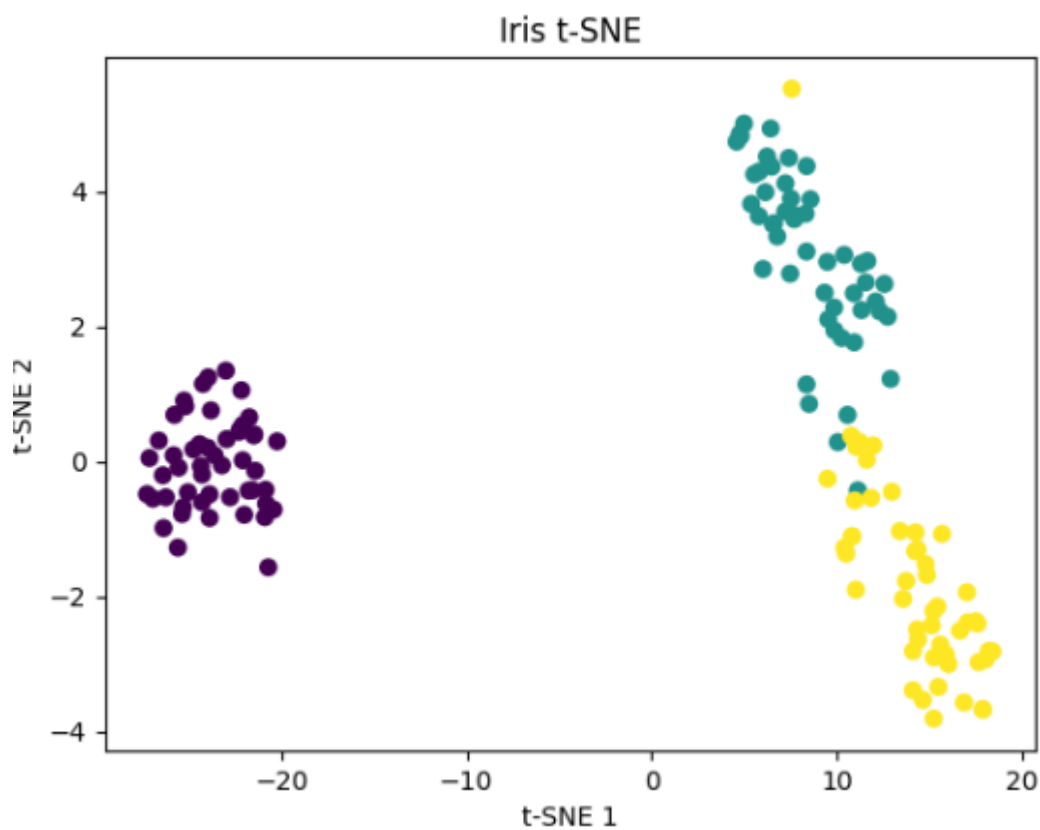
11. Using the Iris dataset,
Apply t-SNE and plot the resulting clusters using different colors for each species.

CODE

```
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

iris = load_iris()
X_tsne = TSNE(
    n_components=2,
    random_state=42
).fit_transform(iris.data)
```

```
plt.scatter(
    X_tsne[:,0],
    X_tsne[:,1],
    c=iris.target
)
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.title("Iris t-SNE")
plt.show()
OUTPUT
```



12. A customer segmentation dataset contains 40 behavioral variables.

Visualize customer groups using t-SNE.

Discuss whether distinct clusters are formed.

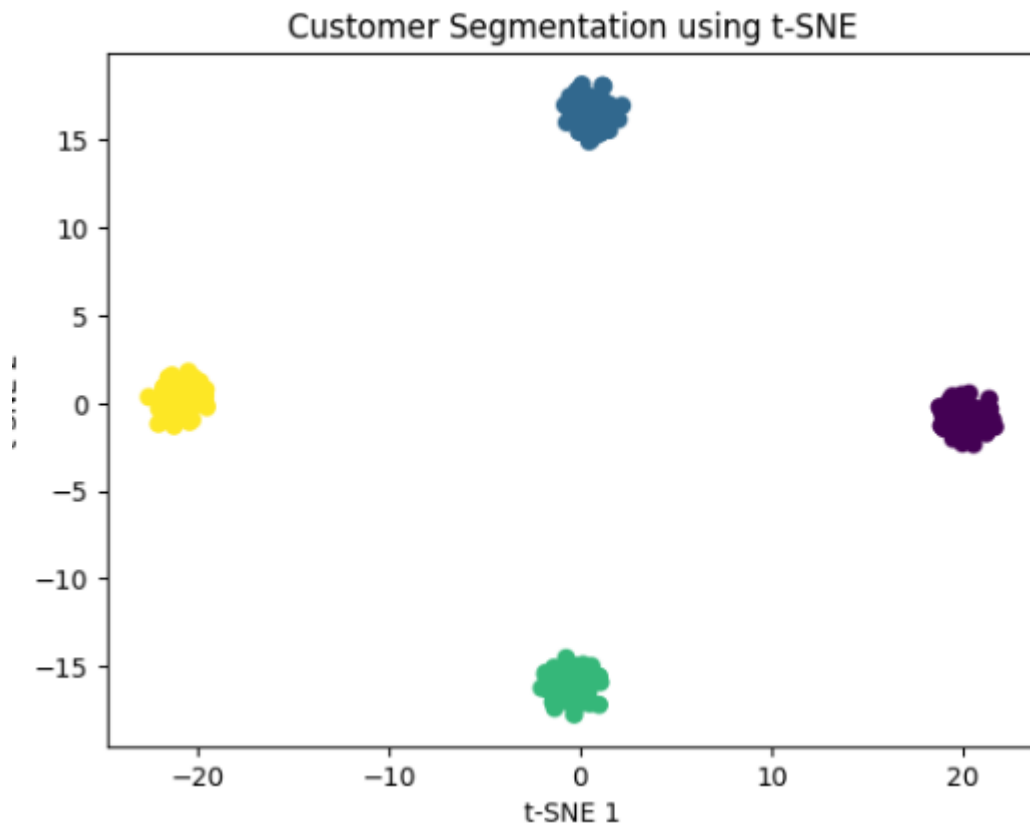
CODE

```
from sklearn.datasets import make_blobs
```

```
from sklearn.manifold import TSNE
```

```
import matplotlib.pyplot as plt
X, y = make_blobs(
    n_samples=200,
    n_features=40,
    centers=4,
    random_state=42
)
X_tsne = TSNE(
    n_components=2,
    random_state=42
).fit_transform(X)
plt.scatter(X_tsne[:,0], X_tsne[:,1], c=y)
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.title("Customer Segmentation using t-SNE")
plt.show()
```

OUTPUT



13. A medical image dataset contains 100 extracted features.

Apply UMAP to reduce the dimensions to two.

Visualize the output.

Compare the result with PCA.

CODE

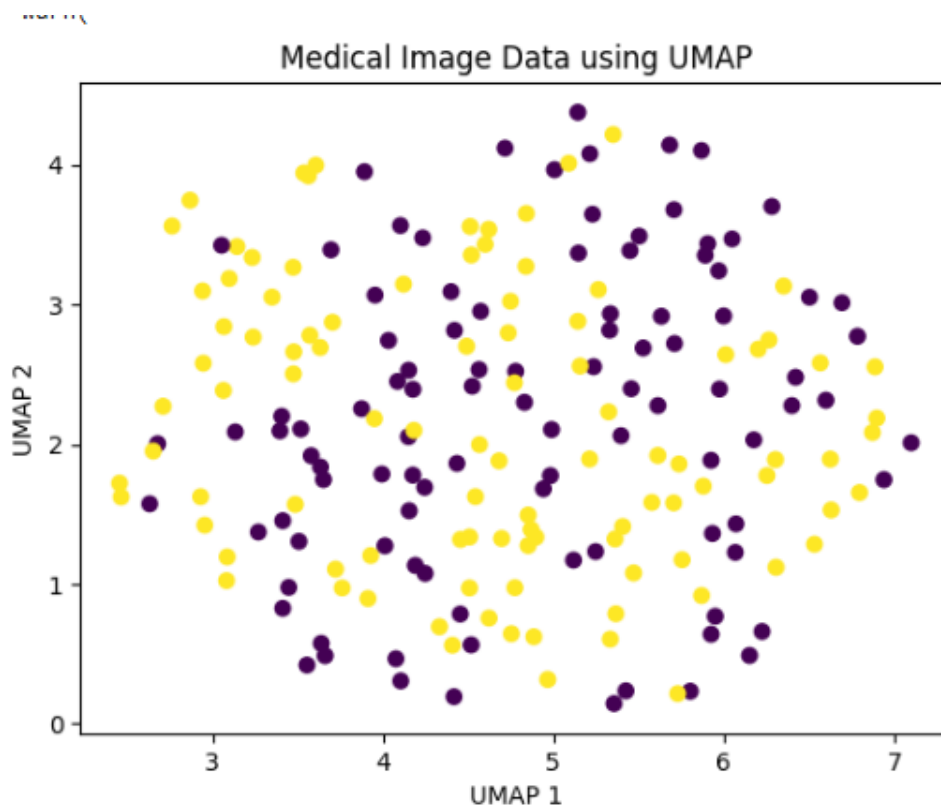
```
import numpy as np
import matplotlib.pyplot as plt
import umap.umap_ as umap
from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
X, y = make_classification(
    n_samples=200,
    n_features=100,
    n_informative=50,
```

```

    random_state=42
)
X = StandardScaler().fit_transform(X)
reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(X)
plt.scatter(X_umap[:,0], X_umap[:,1], c=y)
plt.xlabel("UMAP 1")
plt.ylabel("UMAP 2")
plt.title("Medical Image Data using UMAP")
plt.show()

```

OUTPUT



14. Apply UMAP to the Iris dataset.

Plot the reduced dimensions using different colors for each species.

CODE

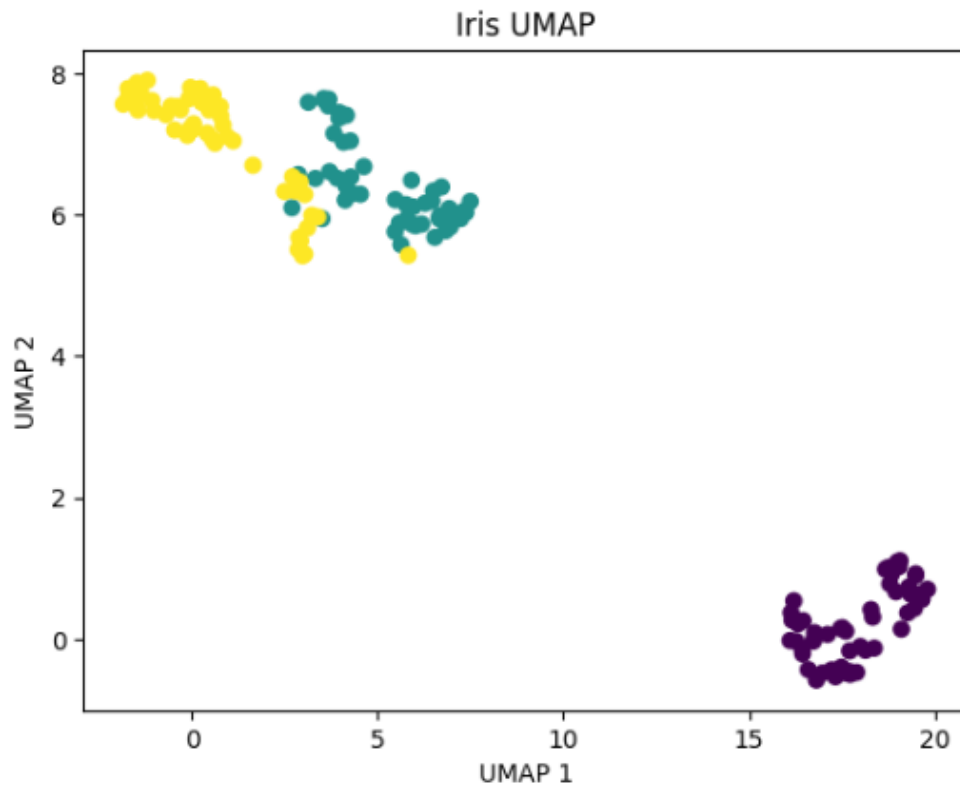
```

from sklearn.datasets import load_iris
import umap.umap_ as umap
import matplotlib.pyplot as plt

```

```
iris = load_iris()
reducer = umap.UMAP(
    n_components=2,
    random_state=42
)
X_umap = reducer.fit_transform(iris.data)
plt.scatter(
    X_umap[:,0],
    X_umap[:,1],
    c=iris.target
)
plt.xlabel("UMAP 1")
plt.ylabel("UMAP 2")
plt.title("Iris UMAP")
plt.show()
```

OUTPUT



15. An e-commerce company has collected customer purchasing behavior consisting of 60 attributes.

Reduce the dimensions using UMAP and identify customer clusters.

CODE

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
import umap.umap_ as umap
import matplotlib.pyplot as plt

X, y = make_blobs(
    n_samples=300,
    n_features=60,
    centers=4,
    random_state=42
)

X = StandardScaler().fit_transform(X)
reducer = umap.UMAP(
```

```
n_components=2,  
random_state=42  
)  
X_umap = reducer.fit_transform(X)  
plt.scatter(  
    X_umap[:,0],  
    X_umap[:,1],  
    c=y  
)  
plt.xlabel("UMAP 1")  
plt.ylabel("UMAP 2")  
plt.title("Customer Clusters using UMAP")  
plt.show()
```

OUTPUT

